MASTERMIND · MARÇO 2026

Como construí minha *equipe* de agentes IA

De um chatbot genérico a um sistema com 10 agentes especializados, memória persistente e operação autônoma 24h.

ISIS MOREIRA · 17 SLIDES

A HISTÓRIA

Como *criei* minha equipe de agentes

Comecei querendo automatizar tarefas repetitivas. Terminei com um time que produz, analisa, cria conteúdo e opera 24 horas sem parar.

A ideia surgiu da dor real: eu fazia tudo sozinha. Copy, design, tráfego, suporte, análise. Cada tarefa delegada para IA genérica voltava genérica. Precisava de agentes que entendessem meu tom, meu público e meu padrão.

Construí em 5 dias. Não 5 meses. Não precisei de desenvolvedor. Não precisei de equipe técnica. Precisei de clareza sobre o que cada "funcionário" deveria fazer.

O ponto de virada

Parei de usar IA como ferramenta e passei a usar como equipe. Cada agente tem nome, função, tom de voz e limites definidos.

Resultado

Materiais que levavam dias saem em minutos. Diagnósticos de clientes, ebooks, PDFs visuais, análise de campanhas. Tudo no meu padrão.

O segredo

Não é a tecnologia. É a arquitetura. Saber o que cada agente faz, o que não faz, e como eles se comunicam entre si.

DE ONDE VIM

N8N vs *OpenClaw*

N8N: onde comecei

Automação visual com nodes. Conectava APIs, disparava emails, processava webhooks. Funciona bem para fluxos lineares: se X acontece, faz Y.

Limitação

Cada fluxo é fixo. Precisa prever todos os cenários. Não aprende, não lembra, não tem personalidade. Cada interação começa do zero.

Manutenção

Workflows quebram silenciosamente. APIs mudam, tokens expiram, e você descobre quando o cliente reclama.

OpenClaw: onde estou

Agentes com identidade, memória e capacidade de decisão. Não é fluxo fixo: o agente entende o contexto e age.

Diferencial

Memória persistente. O agente lembra das decisões, dos erros, do histórico. Cada interação é mais inteligente que a anterior.

Operação

Roda no meu servidor, conectado ao meu Telegram. Sem interface visual para montar fluxos: tudo é linguagem natural + arquivos de configuração.

COMPARATIVO

OpenClaw vs *outras* plataformas

CRITÉRIO	CHATGPT / GEMINI	N8N / MAKE	BOTPRESS / VOICEFLOW	OPENCLAW
Memória persistente	Limitada (contexto de janela)	Sem memória nativa	Variáveis de sessão	**Arquivos persistentes no servidor**
Múltiplos agentes	1 por conversa	1 por workflow	1 bot por projeto	**10+ agentes no mesmo sistema**
Identidade/personalidade	Custom instructions básico	Não tem	Prompt estático	**SOUL.md, IDENTITY.md, tom de voz**
Executa código	Sandbox limitado	Sim (nodes)	Limitado	**Acesso total ao servidor (bash, Python)**
Conecta no Telegram/WhatsApp	Não nativo	Via API	Sim	**Nativo, multi-canal**
Custo mensal	US$20-200	US$0-99	US$0-300	**~US$25 (servidor + Claude)**
Controle total	Não (plataforma deles)	Parcial	Não	**100% no seu servidor**

INVESTIMENTO

Quanto custa manter essa *equipe*

Menos que um estagiário. Mais produtivo que uma equipe de 5.

R$30 SERVIDOR / MÊS	~R$100 CLAUDE API / MÊS	R$0 OPENCLAW
~R$5 WHISPER API	R$0 GEMINI (IMAGENS)	~R$135 TOTAL / MÊS

Servidor: Hostinger VPS
KVM 2, 8GB RAM, 100GB SSD. Roda Ubuntu 24.04. Suficiente para todo o sistema + processamento.

LLM: Claude (Anthropic)
Modelo Opus para tarefas complexas, Haiku para heartbeats e tarefas leves. Pay-per-use, sem mensalidade fixa.

OpenClaw: open source
Gratuito. Instala no servidor, configura uma vez, roda indefinidamente. Atualizações manuais quando eu quiser.

EVOLUÇÃO

Minha *jornada* no OpenClaw

Servidor VPS | Instalar OpenClaw | Bot no Telegram | Agente Dona | Time de 10 agentes

Dia 1: Servidor
Contratei um VPS na Hostinger. Ubuntu limpo. Instalei o OpenClaw via linha de comando. 20 minutos.

Dia 1: Bot do Telegram
Criei um bot pelo BotFather, conectei ao OpenClaw. Primeira conversa com a "Dona" direto no meu celular.

Dia 2: Identidade
Defini SOUL.md (personalidade), IDENTITY.md (quem ela é), USER.md (quem eu sou). A Dona deixou de ser genérica.

Dia 3: Supergrupo + Tópicos
Criei um grupo no Telegram com tópicos separados. Cada tópico virou um agente diferente. Mesma IA, múltiplas identidades.

Dia 4: Skills + Ferramentas
Instalei skills: geração de imagem, transcrição de áudio, conversão de PDF. A equipe ganhou braços.

Dia 5: Produção real
Primeiro material gerado pela cadeia completa: Masterson escreveu, Aurora formatou, virou PDF. Em 3 minutos.

ARQUITETURA

O *organograma* da equipe

O TIME

Cada agente tem sua *função*

Dona
COO

Coordena o time. Analisa dados. Executa estratégia. Fala direto comigo.

Atlas
ESTRATEGISTA

Planeja antes de executar. Gerencia risco. Orquestra cadeias com 2+ agentes.

Masterson
ESCRITOR

Ebooks, páginas de vendas, sequências de email, scripts de VSL, materiais longos.

Vega
SOCIAL MEDIA

Posts, carrosséis, legendas, pesquisa de tendências, scraping de perfis.

Aurora
DESIGNER

PDFs, ebooks visuais, apresentações. Aplica o Design System da marca.

Harpia
VÍDEO

Roteiros, briefings de edição, direção criativa de conteúdo audiovisual.

Helios
TRÁFEGO

Campanhas Meta Ads, análise de métricas, otimização de ROAS, diagnósticos.

Nexa
IMAGEM

Gera fotos editoriais com IA. Banco de imagens consistente com a marca.

Orion
DEV

Código, automações, scripts, integrações, manutenção técnica do sistema.

Eva
ARQUITETA

Cria e evolui os próprios agentes. Define identidades e documentação.

FERRAMENTAS

Skills: os braços da equipe

Skills são ferramentas que os agentes podem usar. Instalam no servidor e ficam disponíveis para qualquer agente que precise.

Nano Banana Pro
Geração de imagens via Gemini. Nexa usa para criar fotos editoriais da marca com IA.

Whisper API
Transcrição de áudio. Converte gravações, lives e mentorias em texto pesquisável.

Video Frames
Extrai frames de vídeo com ffmpeg. Analisa thumbnails e momentos de reels.

Nano PDF
Gera PDFs a partir de HTML. Aurora usa para produzir materiais visuais no padrão da marca.

Como instalar uma skill
Um comando no terminal: openclaw skills install nome-da-skill. Pronto. O agente já pode usar.

Skills vs APIs manuais
Skills vêm prontas. Sem configurar endpoints, headers, autenticação. O agente chama e a skill resolve.

Comunidade
Repositório público de skills no ClawhHub. Qualquer pessoa pode criar e compartilhar skills.

NA PRÁTICA

Usos e *aplicações* reais

Diagnóstico para clientes

Recebo as métricas de uma campanha, peço ao Atlas. Ele usa Helios (análise) + Masterson (copy) + Aurora (design) e entrega um PDF de 7 páginas com diagnóstico completo e plano de ação. Em minutos.

Framework de distribuição de tráfego

Classifico posts em Série A (escalar) e Série B (manter) baseado no custo por seguidor. Helios monitora diariamente e realoca verba automaticamente.

Ebooks e materiais visuais

Masterson escreve o conteúdo. Aurora aplica o Design System e gera o PDF final. 40+ páginas em menos de uma hora, no padrão da minha marca.

Resumo de mentoria

Gravo a sessão, mando o áudio. Whisper transcreve, Masterson organiza em tópicos, Aurora formata. Entrego para o aluno um material estruturado.

Transcrição de cursos

Pipeline para transcrever todos os meus cursos: download do Vimeo, extração de áudio, transcrição, organização por módulo. Vira base de conhecimento.

Geração de fotos editoriais

Nexa gera fotos minhas com IA. DNA fixo (estilo, câmera, iluminação) + variações controladas (pose, roupa, cenário). Banco de imagens infinito.

ESTRUTURA

Anatomia de um agente

Cada agente é definido por arquivos de texto. Não precisa de código. É linguagem natural estruturada.

SOUL.md
A personalidade. Tom de voz, como responde, o que faz e o que não faz. Define quem o agente É.

IDENTITY.md
Nome, papel, email, nível de autonomia. O cartão de visita do agente.

AGENTS.md
Mapeamento de identidades por tópico. Define quem é quem no sistema e as regras de cada um.

USER.md
Quem é o dono. Nome, preferências, horário, regras de comunicação. O agente sabe para quem trabalha.

MEMORY.md
Índice da memória. Aponta para decisions.md, lessons.md, projects.md, pending.md. O agente sabe o que já aconteceu.

HEARTBEAT.md
Checklist periódico. O que verificar a cada pulso: pendências, projetos parados, notas não consolidadas.

TOOLS.md
Ferramentas disponíveis, caminhos de binários, APIs configuradas. O manual técnico do agente.

knowledge/
Pasta com base de conhecimento específica. Cada agente tem seus materiais de referência.

AUTOMAÇÃO

Crons e Heartbeats

O sistema não espera eu pedir. Ele age sozinho nos horários programados.

Cron: tarefa agendada

Programo uma tarefa para rodar em horários específicos. Exemplo: "todo dia às 8h, analise minhas métricas e me mande um resumo". O agente executa sem eu pedir.

Heartbeat: pulso de verificação

A cada intervalo, o sistema "acorda" e verifica: há pendências atrasadas? Projetos parados? Crons que falharam? Notas não consolidadas? Se algo precisa de atenção, me avisa. Se não, volta a dormir.

Na prática

Acordo de manhã e já tenho: análise de conteúdo, métricas de vendas, tendências do nicho, insights estruturados. Tudo em bullet points, sem precisar pedir.

Exemplos de crons

- [ ] Análise diária às 08h
- [ ] Verificação de crons falhados
- [ ] Consolidação semanal de notas
- [ ] Revisão de projetos (segunda)
- [ ] Limpeza de memória (mensal)

O que o heartbeat checa

- [ ] Pendências com mais de 3 dias
- [ ] Projetos parados há 7+ dias
- [ ] Crons que não rodaram
- [ ] Notas diárias não consolidadas

MANUTENÇÃO

Higiene do sistema

IA sem manutenção vira bagunça. Assim como uma equipe real precisa de processos, agentes precisam de higiene.

Compactação de contexto

Conversas longas são compactadas para não estourar o limite de tokens. Antes de compactar, o sistema extrai decisões, lições e pendências para arquivos permanentes.

Consolidação de notas

Notas diárias são consolidadas em topic files (decisions.md, lessons.md, projects.md). A cada poucos dias, o sistema organiza o que aprendeu.

Limpeza mensal

Lições táticas com mais de 30 dias são revisadas e deletadas se não forem mais relevantes. Memória limpa = respostas melhores.

Por que importa

Sem higiene, o agente começa a repetir erros antigos, confundir contextos e ficar lento. Memória acumulada sem organização é lixo, não inteligência.

Regra de extração

Antes de qualquer compactação: extrair decisões, lições, atualizar projetos, atualizar pendências, reindexar memória. Nessa ordem. Sempre.

Indexação

Após cada mudança nos memory files: openclaw memory index. Isso reconstrói o índice semântico para buscas rápidas.

MONITORAMENTO

Como funcionam os *alertas*

O sistema monitora a si mesmo. Quando algo está errado, me avisa antes que eu perceba.

Tópico Alertas (dedicado)

Tenho um tópico no Telegram só para alertas do sistema. A Dona monitora e avalia cada um. Correções operacionais: executa sozinha. Correções que afetam conteúdo público: me pede aprovação.

Formato padronizado

[ALERTA] Tipo: heartbeat_check · Status: atenção · Hora: 08:00 UTC · Detalhe: 2 pendências com mais de 3 dias sem resposta.

Gates de decisão

Gate verde: o agente corrige sozinho e registra. Gate vermelho: para e me pede ok. Nada que afete conteúdo público sai sem minha aprovação.

O que gera alerta

- [ ] Cron falhou ou não rodou
- [ ] Pendência atrasada (3+ dias)
- [ ] Projeto parado (7+ dias)
- [ ] Erro no gateway/servidor
- [ ] Token expirado
- [ ] Memória não indexada
- [ ] Anomalia em métricas

O sistema cuida de si mesmo. Eu cuido da estratégia.

PERSISTÊNCIA

Memória do sistema

Diferente de chatbots comuns, meus agentes lembram. Decisões, erros, preferências, histórico. Tudo persiste entre sessões.

Conversa acontece → Nota diária criada → Consolidada em topic files → Indexada para busca

decisions.md
Decisões permanentes. O que foi decidido, quando, e por quê. Nunca se perde.

lessons.md
Lições aprendidas. Erros que não devem se repetir. Padrões descobertos.

projects.md
Projetos ativos, em andamento e concluídos. Status sempre atualizado.

pending.md
Aguardando input meu ou de terceiros. Nada cai no esquecimento.

Busca semântica
O agente não precisa lembrar de tudo. Ele busca na memória por significado, não por palavra exata. "Qual foi a decisão sobre tráfego?" encontra o registro mesmo sem a palavra "tráfego" no título.

Feedback loops
Arquivos JSON que registram feedback por domínio: conteúdo, tarefas, recomendações. O sistema aprende com cada correção minha.

Resultado prático
Não preciso repetir contexto. O agente sabe quem são minhas alunas, qual meu tom de voz, quais decisões já tomei, o que está pendente. Cada interação é continuação, não recomeço.

INTELIGÊNCIA

Base de *conhecimento* e limite dos LLMs

Agentes são tão bons quanto o conhecimento que têm. Sem base, é IA genérica. Com base, é especialista.

Knowledge base por agente

Cada agente tem uma pasta knowledge/ com materiais de referência. Masterson tem livros de copywriting. Helios tem documentação do Meta Ads. Aurora tem o Design System. Eles consultam antes de responder.

Contexto limitado = decisão estratégica

LLMs têm janela de contexto (200k tokens no Claude). Não cabe tudo de uma vez. Por isso a base é organizada: o agente busca o que precisa, quando precisa, via memória semântica.

Modelo certo para cada tarefa

Opus (caro, preciso) para tarefas complexas: estratégia, copy, análise. Haiku (barato, rápido) para heartbeats e verificações. Gemini para imagens. Whisper para áudio. Cada centavo no lugar certo.

Bases montadas

- [ ] Masterson: 5 livros de copywriting
- [ ] Helios: 4 docs Meta Ads
- [ ] Aurora: Design System ISIS v7
- [ ] Nexa: 26 poses, 26 roupas, 23 cenários
- [ ] Todos: USER.md (tom de voz de Isis)

Próximas bases

- [ ] Transcrição de todos os cursos
- [ ] Livro "Acorda! Sonhar não Basta"
- [ ] Histórico de campanhas reais
- [ ] FAQs do suporte

PRÓXIMO PASSO

Bot *Donazinha*

O objetivo final: um bot que responde meus mentorandos com a mesma profundidade que eu responderia.

O que é

Um bot treinado com todo o conteúdo dos meus cursos, mentorias e materiais. Responde dúvidas dos alunos com base no que eu já ensinei, no meu tom de voz, com referências aos módulos específicos.

Como funciona

Aluno pergunta no grupo ou DM. Donazinha busca na base de conhecimento (transcrições, FAQs, materiais). Responde com a minha voz, cita o módulo/aula relevante, e indica onde o aluno pode se aprofundar.

Por que muda o jogo

Suporte 24h sem eu estar presente. Alunos recebem respostas de qualidade a qualquer hora. Eu ganho tempo para estratégia e conteúdo novo. Marlua e Érica ganham um assistente que sabe tudo que eu já ensinei.

Pipeline de construção

1. Transcrever todos os cursos
2. Organizar por módulo/tema
3. Gerar FAQs do suporte
4. Treinar base de conhecimento
5. Configurar bot no Telegram
6. Testar com grupo piloto
7. Ativar para todos os alunos

Uma extensão de mim que nunca dorme, nunca cansa e nunca esquece o que eu já ensinei.